MiniCV Library

Milestone 1 Technical Report

Design & Implementation of a From-Scratch Computer Vision Library

---

Course: Computer Vision

Milestone: 1

Date: April 2026

Prepared by: [Mohammed Mahmoud AlJamal
& Abdelrahman Khaled Elsayed]

1. Introduction

The MiniCV library is a Python language-based image processing package developed entirely from the ground up, with only NumPy for numerical computations, Pandas for tabular data structures, and Matplotlib for visualization and image input/output (I/O). This library meant to provide a development in computer vision field without specialized image-processing dependencies but can cover a significant portion of the OpenCV computer-vision library's functions. This "constraint" (non-dependency on any special image-processing packages) is not a restriction for its pedagogical and engineering aim to prove that robust, production-quality computer vision routines can be built entirely from mathematical first-principles and performant array computation. The motivation to reimplement classical vision algorithms entirely from scratch serves multiple purposes. First, it requires a deep understanding of the related mathematics behind the image processing operations — convolution kernels, geometric transformation matrices, histogram probability distributions, and gradient operators — rather than blindly wrapping these functionalities with library calls. If a practitioner reimplements median filtering from scratch using raw NumPy, he understands why such a filter requires more computation than mean filtering, how boundary conditions impact the output quality, and possible trade-offs between precision and speed. Second, there is a transparent, auditable codebase with no hidden abstractions to hinder experimentation or extension. Third, the implementations are driven by rigorous software engineering discipline: modular decomposition, input validation, vectorized implementations, and the public API are not negotiable when there is no external library to delegate the complexity to. Milestone 1 details the fabric of MiniCV: image I/O, core numerical operations, a 2D convolution engine, spatial and photometric filtering, geometric transformations, primitives for basic feature extractions, and rasterized drawing utilities. That is sufficient for the preprocessing and feature preparation pipelines that will feed the machine learning elements planned in Milestone 2. Architectural decisions made in Milestone 1 are therefore significant downstream and have been made with extensibility as an explicit concern.

2. System Architecture

2.1 Package Structure

MiniCV is organized as a Python package with a flat, module-per-responsibility hierarchy. The top-level namespace exposes a curated public API, while each subpackage corresponds to a distinct functional domain. The directory layout is reproduced below for reference:

```
minicv/
foundation_functions.py
filters.py
geometric_transformation.py
features.py  drawing.py
writing.py  image_io.py
```

This layout was chosen to mirror the organizational conventions of mature computer vision libraries while keeping the dependency graph acyclic. each module has a clearly defined responsibility and minimal dependencies on other modules, preventing circular imports and simplifying testing, never on sibling subpackages. This discipline eliminates circular imports and makes it trivial to unit-test any module in isolation.

2.2 Dependency Constraints

This package follows strict external-dependency limitation policies, where Numpy, Pandas, Matplotlib, and Python standard libraries are the only allowed imports. There are several important engineering consequences to the constraint. Array manipulation, kernel arithmetic, and pixel-wise operations all have to be written as vectorized “NumPy” operations. Loops are allowed only when the algorithm is inherently sequential (for example, Bresenham’s line rasterization) or where a vectorized implementation would compromise readability too much.

2.3 Engineering Decisions

Various engineering choices have influenced the design of this architecture. Firstly, images are always kept in (H, W, C) format, conforming to the standard used in nearly all modern deep learning libraries where channel-wise operations become extremely simple. The grayscale images have (H, W) shape, whereas their promotion to (H, W, 1) is done exclusively when convolution engine demands it. Secondly, all functions take and produce NumPy arrays of float32 data type to prevent overflow issues in computation; clipping and conversion to uint8 data type happens strictly at the output stage. Lastly, convolution engine serves as the sole canonical implementation for all types of filtering; accordingly, mean and Gaussian filter functions simply construct kernels and pass them into the same engine pipeline.

3. Mathematical Foundations

3.1 Normalization

Image normalization maps pixel values from an arbitrary range into a target range. The canonical min-max normalization is defined as:

$$I_norm(x,y) = (I(x,y) - I_min) / (I_max - I_min)$$

where I_min and I_max are the global minimum and maximum pixel values respectively. A scaled variant maps into [a, b] by applying a further linear transformation. Zero-mean unit-variance (Z-score) normalization subtracts the channel mean and divides by the standard deviation, which is preferred when the downstream consumer expects zero centered input such as a neural network. The MiniCV implementation supports all three modes through a single entry-point function parameterized by a mode string.

3.2 Discrete 2D Convolution

The discrete 2D convolution of an image I with a kernel K is:

```
(I * K)(x,y) = SUM_m SUM_n I(x+m, y+n) * K(m, n)
```

where the summation runs over the spatial extent of the kernel. Strictly speaking, this is a cross-correlation in the signal-processing sense; true convolution requires the kernel to be flipped both horizontally and vertically. For symmetric kernels such as Gaussian and mean filters the distinction is immaterial, but the implementation accepts a flip_kernel boolean flag to handle asymmetric kernels correctly. Boundary handling is controlled by the padding mode argument, which selects between zero-padding, reflectpadding, and replicate-padding prior to convolution, avoiding the need for conditional logic inside the inner loop.

The implementation vectorizes the operation using NumPy's stride_tricks.as_strided to extract a view of all kernel-sized patches simultaneously, then contracts them against the kernel weights with a single dot product or einsum call. This achieves near-optimal cache utilization and avoids any Python-level iteration over spatial positions.

3.3 Gaussian Filtering

The Gaussian kernel of size k x k with standard deviation sigma is defined by:

```
G(m, n) = (1 / (2*pi*sigma^2)) * exp(-(m^2 + n^2) / (2*sigma^2))
```

After evaluation on the integer grid, the kernel is normalized so that its coefficients sum to unity, which preserves the mean brightness of the image. Gaussian filtering is the separable linear smoother that minimizes the space-bandwidth product: no other filter of the same bandwidth achieves superior noise suppression with fewer multiplications. The 2D kernel is separable into two 1D kernels, so full separable filtering requires only 2k multiplications per pixel versus k^2 for the direct 2D application. MiniCV implements both paths; the separable route is selected automatically when the kernel is recognized as a Gaussian.

3.4 Median Filtering

Median filtering replaces each pixel with the statistical median of all pixel values within a k x k neighbourhood. Because the median is a nonlinear order-statistic rather than a linear inner product, it cannot be decomposed into a kernel convolution. Formally:

```
I_med(x,y) = median{ I(x+m, y+n) : -k/2 <= m,n <= k/2 }
```

The median operation is particularly effective at removing salt-and-pepper noise because it substitutes extreme outlier values with the central tendency of the neighbourhood without blurring edges, a property that Gaussian filtering does not possess. Computationally, extracting sorted medians over a sliding window requires O(H * W * k^2 * log(k)) operations in the naive implementation. MiniCV uses a vectorized patch-extraction approach followed by numpy.median applied along the patch axis, achieving performance adequate for moderate-resolution images.

3.5 Sobel Gradient Operators

The Sobel operator approximates the image gradient using two 3x3 kernels that compute first-order derivatives in the horizontal and vertical directions:

```
K_x = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]
K_y = [[-1,-2,-1], [ 0, 0, 0], [ 1, 2, 1]]
```

The gradient magnitude and orientation at each pixel are then:

```
G = sqrt(G_x^2 + G_y^2) theta
  = arctan2(G_y, G_x)
```

The Sobel kernels perform simultaneous differentiation and Gaussian smoothing, making them more robust to noise than the simpler Prewitt kernels. The gradient magnitude image highlights edges and is a necessary precursor to many feature extraction pipelines including the gradient orientation histogram descriptor implemented in the features module.

3.6 Histogram and Histogram Equalization

The normalized histogram of an 8-bit grayscale image is an estimate of the probability mass function of pixel intensities:

```
p(k) = n_k / (H * W),  for k in {0, 1, ..., 255}
```

where n_k is the count of pixels with intensity k and H x W is the total pixel count. Histogram equalization seeks a monotonic mapping T(k) that transforms the original distribution toward a uniform distribution, thereby maximising image contrast. The cumulative distribution function (CDF) of the histogram provides this mapping:

```
T(k) = round( (L - 1) * CDF(k) )
```

where L is the number of grey levels (256 for uint8 images). The operation is implemented in MiniCV as a NumPy lookup-table transform: the mapping array T is computed once from the histogram CDF and then applied to the entire image array via fancy indexing, giving O(H*W) time complexity after the O(L) preprocessing step.

3.7 Thresholding

Thresholding converts a grayscale image to a binary image by comparing each pixel to a threshold value tau:

```
I_bin(x,y) = 255 if I(x,y) >= tau else 0
```

The choice of tau is non-trivial for arbitrary images. Otsu's method determines the globally optimal threshold by maximizing the between-class variance of foreground and background:

```
sigma_B^2(tau) = w_0(tau) * w_1(tau) * (mu_0(tau) - mu_1(tau))^2
```

where w_0, w_1 are class probabilities and mu_0, mu_1 are class means. MiniCV implements both manual thresholding and automatic Otsu thresholding as separate functions within the filters module.

3.8 Geometric Transformations

All geometric transformations in MiniCV are implemented using inverse mapping: for each output pixel coordinate (x', y'), the corresponding source coordinate (x, y) is computed by applying the inverse transformation matrix, and the output value is determined by interpolating the source image at that coordinate. This strategy avoids holes in the output image that arise from forward mapping when the transformation is not surjective.

For translation by (tx, ty), the inverse map is simply (x, y) = (x' - tx, y' - ty). For rotation by angle theta about the image center (cx, cy), the inverse map is:

```
x = (x' - cx)*cos(theta) + (y' - cy)*sin(theta) + cx
y = -(x' - cx)*sin(theta) + (y' - cy)*cos(theta) + cy
```

3.9 Interpolation

Two interpolation methods are provided. Nearest-neighbour interpolation assigns to each output pixel the value of the closest source pixel by rounding the fractional source coordinates to the nearest integer. It is fast, lossless in terms of intensity range, and appropriate for categorical label images. Bilinear interpolation computes a weighted average of the four surrounding source pixels:

```
I(x, y) = (1-a)(1-b)*I(x0,y0) + a(1-b)*I(x1,y0) + (1-a)*b*I(x0,y1) +
                                a*b*I(x1,y1)
```

where (x0, y0) = (floor(x), floor(y)), x1 = x0+1, y1 = y0+1, a = x - x0, b = y - y0. Bilinear interpolation is C0 continuous and produces noticeably smoother results than nearestneighbour at the cost of two extra memory accesses and four multiplications per pixel.

3.10 Gradient-Based Descriptors

The gradient orientation descriptor aggregates Sobel-derived gradient magnitudes and orientations into a compact fixed-length representation. The gradient direction at each pixel is computed as:

$$\theta = \tan^{-1}(G_y/G_x)$$

The continuous orientation range $[0°, 180°]$is discretized into Bbins, and each pixel contributes to the bin corresponding to its orientation, weighted by the gradient magnitude:

$$h_k = \sum_{(x,y)\in bin_k} G(x, y)$$

where $G(x, y) = \sqrt{G_x^2 + G_y^2}$.

This produces a normalized histogram describing the dominant edge directions present in the image. While conceptually related to the Histogram of Oriented Gradients (HOG), the implementation in MiniCV uses a global histogram rather than a spatial cell grid, resulting in a compact descriptor that captures overall structural information.

4. Implementation Details

4.1 Image I/O

Image reading is implemented using Matplotlib's pyplot.imread function, which decodes JPEG, PNG, BMP, and TIFF files into NumPy arrays of shape (H, W, C) with dtype uint8 or float32 depending on the file format. Upon loading, MiniCV converts all arrays to float32 and normalizes pixel values to the [0, 1] range, establishing a canonical internal representation that is independent of the source file format. Writing converts float32 arrays back to uint8 by multiplying by 255 and clipping, then invokes pyplot.imsave.

Grayscale conversion follows the luminosity-weighted formula recommended by the ITU-R BT.601 standard: Y = 0.299*R + 0.587*G + 0.114*B. This weighting reflects the differential sensitivity of human photoreceptors to red, green, and blue light. The conversion is implemented as a single NumPy dot product along the channel axis, ensuring efficiency for large images.

4.2 Core Operations

The padding module implements three strategies that are accessed identically by all downstream consumers. Zero padding appends rows and columns of zeros around the image border. Reflect padding mirrors the image content across each boundary, which reduces ringing artefacts in frequency-sensitive applications. Replicate (edge) padding repeats the outermost row and column values, which is appropriate when edge continuation is a more accurate prior than zero extension. The padding size is inferred automatically from the kernel dimensions passed to the convolution function, so callers are never required to manage padding manually.

The validation module provides a centralized set of guard functions that raise ValueError or TypeError with descriptive messages when inputs violate expected contracts. Functions validated include: checking that image arrays are two- or threedimensional, that kernel sizes are odd positive integers, that dtype is numeric, and that target dimensions for resizing are positive integers. Centralizing these checks eliminates duplication and ensures consistent error messaging across the library.

4.3 Convolution Engine

The convolution engine found in minicv/core/convolution.py is the module that has the highest requirements regarding performance within the library. This implementation is done in three steps. At first, the image is padded depending on the padding method chosen. Then, using numpy.lib.stride_tricks.sliding_window_view (and as_strided in case of older NumPy versions), all patches of size k*k are assembled into a tensor of shape (H_out, W_out, k, k). Finally, the operation is performed by contracting this tensor with a flattened version of the kernel with numpy.einsum('hwij,ij->hw', patches, kernel), which is optimized to a BLAS SGEMM call, delivering peak floating-point performance. Multi-channel images are processed by looping over channels, which is the only loop in the code that is not manually vectorized. However, since natural images do not usually have more than three or four channels, the effect on performance is negligible.

4.4 Filtering Pipeline

Each filter in the filters subpackage is implemented as a kernel factory paired with a call to the shared convolution backend. The mean filter constructs a k x k array of uniform values summing to one. The Gaussian filter calls a dedicated gaussian_kernel(size, sigma) function that evaluates the Gaussian formula on an integer meshgrid, then normalizes. Both are then passed directly to convolve2d, ensuring that any improvement to the convolution engine benefits all filters simultaneously. The median filter cannot follow this pattern and instead constructs the patch tensor explicitly before applying numpy.median along the patch axes.

4.5 Transformations

All three geometric transformations (resize, rotate, translate) follow a common inversemapping template. A meshgrid of output pixel coordinates is constructed, the

inverse transformation is applied in closed form using vectorized arithmetic to produce corresponding source coordinates, and interpolation is performed using one of the two registered interpolation functions. The entire pipeline operates without any explicit Python loops over pixel positions, relying instead on NumPy broadcasting and advanced indexing. For bilinear interpolation, floor and ceiling indices are clipped to the valid image range using numpy.clip to handle boundary cases gracefully.

4.6 Feature Extraction

The feature extraction module computes a compact set of global image descriptors designed to capture both photometric and structural characteristics of grayscale images. All descriptors are implemented using vectorized NumPy operations and returned as a single one-dimensional float32 feature vector suitable for downstream machine learning tasks.

The implemented descriptors include:

• Mean intensity descriptor
The global average pixel intensity provides a simple measure of overall brightness:

$$\mu = \frac{1}{N}\sum_{x,y} I(x,y)$$

• Intensity histogram descriptor
A normalized histogram of grayscale values is computed using a fixed number of bins (default = 16). The histogram approximates the probability distribution of pixel intensities:

$$p(k) = \frac{n_k}{N}$$

where $n_k$is the number of pixels whose intensity falls inside bin k.

• Edge density descriptor
The proportion of pixels with significant gradient magnitude is computed using Sobel operators:

$$G = \sqrt{G_x^2 + G_y^2}$$

Pixels whose normalized gradient magnitude exceeds a threshold are classified as edges, and the edge density is defined as:

$$\rho = \frac{N_{edges}}{N_{pixels}}$$

• Gradient orientation histogram descriptor

Gradient orientations are quantized into a fixed number of bins (default = 8) over the range $[0°, 180°]$. Each pixel contributes to the histogram proportionally to its gradient magnitude:

$$h_k = \sum_{(x,y) \in bin_k} G(x, y)$$

The final feature vector is constructed by concatenating all descriptors:

$$d = 1 + 16 + 1 + 8 = 26$$

Where:

descriptor	dimension
mean intensity	1
histogram bins	16
edge density	1
orientation histogram	8

This compact representation captures both global brightness statistics and structural edge information while maintaining low computational complexity.

4.7 Drawing Primitives

The drawing module provides rasterization of basic geometric shapes directly onto NumPy arrays. Points are set by direct index assignment. Lines are rasterized using Bresenham's line algorithm, which iterates over the dominant axis and computes the corresponding minor-axis coordinate using integer arithmetic, avoiding floating-point rounding for every pixel. The algorithm handles all octants by normalizing the direction vector before entering the main loop. Rectangles are drawn as four line segments. Polygons are drawn as a closed sequence of line segments connecting successive vertices.

An important design decision is that all drawing functions operate in-place, modifying the passed image array rather than returning a copy. This is consistent with the convention used by OpenCV and avoids a potentially expensive memory allocation for large images. Functions accept both colour tuples and scalar grey values, and the colour is broadcast correctly to all channels.

5. API Design

The following section documents representative functions from each major module. All functions follow consistent conventions: images are passed as NumPy float32 arrays in (H, W) or (H, W, C) layout; kernel sizes are expressed as odd positive integers; optional parameters have documented defaults; and error conditions are signalled via standard Python exceptions with descriptive messages.

5.1 minicv.io.read_image

Function	`read_image(path, as_gray=False)`
Parameters	path: str - Filesystem path to the image file. as_gray: bool - If True, converts to single-channel grayscale on load.
Returns	np.ndarray of dtype float32, shape (H, W, C) or (H, W) for grayscale.
Error Handling	FileNotFoundError if path does not exist. ValueError if file format is unsupported.
Usage	`img = read_image('photo.jpg', as_gray=False)`

5.2 minicv.filters.gaussian_filter

Function	`gaussian_filter(image, kernel_size=5, sigma=1.0, pad_mode='reflect')`

Parameters	image: np.ndarray - Input image (float32). kernel_size: int - Odd integer specifying kernel width and height. sigma: float - Standard deviation of the Gaussian. pad_mode: str - One of 'zero', 'reflect', 'replicate'.
Returns	np.ndarray of same shape and dtype as input.
Error Handling	ValueError if kernel_size is even or non-positive. ValueError if sigma <= 0.
Usage	`smooth = gaussian_filter(img, kernel_size=7, sigma=1.5)`

5.3 minicv.transforms.rotate

Function	`rotate(image, angle_deg, interpolation='bilinear', expand=False)`
Parameters	image: np.ndarray - Input image. angle_deg: float - Clockwise rotation angle in degrees. interpolation: str - 'nearest' or 'bilinear'. expand: bool - If True, expands output canvas to fit entire rotated image.
Returns	np.ndarray of same dtype as input, shape (H, W, C) or expanded shape.
Error Handling	ValueError if interpolation is not a recognised mode string.
Usage	`rotated = rotate(img, 45.0, interpolation='bilinear')`

5.4 minicv.features. gradient_orientation_descriptor

Function	`gradient_orientation_descriptor(image, bins=8)`
Parameters	image: np.ndarray - Grayscale image patch (float32). int - Number of orientation bins used to quantize gradient directions. num_bins: int - Number of angular orientation bins.
Returns	np.ndarray of shape (bins,) dtype float32.
Error Handling	ValueError if image is not 2D or if cell_size exceeds image dimensions.
Usage	`desc = gradient_orientation_descriptor(image, bins=8)`

5.5 minicv.drawing.draw_line

Function	`draw_line(image, pt1, pt2, color, thickness=1)`
Parameters	image: np.ndarray - Image array, modified in-place. pt1, pt2: tuple(int, int) - Start and end pixel coordinates (row, col). color: tuple or scalar - Pixel colour value(s). thickness: int - Stroke width in pixels.
Returns	None (modifies image in-place).
Error Handling	ValueError if coordinates are not integer 2-tuples. TypeError if image is not a writable array.
Usage	`draw_line(img, (10,10), (200,300), color=(1.0, 0.0, 0.0))`

6. Validation and Testing

Correctness of each module was verified through a combination of visual inspection, cross-validation against known analytical results, and edge-case stress testing. No automated test suite framework (such as pytest) was used as an external dependency, but all validation scripts are included in the repository under the tests/ directory.

6.1 Convolution Correctness

The convolution engine was validated by applying the identity kernel (a single 1 at the center surrounded by zeros) and confirming that the output was numerically identical to the input up to floating-point epsilon. A known-value test was then applied using a 3x3 mean kernel on a synthetic checkerboard array with analytically computable expected output. The maximum absolute error between computed and expected outputs was verified to be below 1e-6 for all tested configurations.

Figure 1: Convolution validation. Left: original image. Middle: output after identity kernel showing no change. Right: mean filter applied to checkerboard image with analytically expected smoothing behavior.

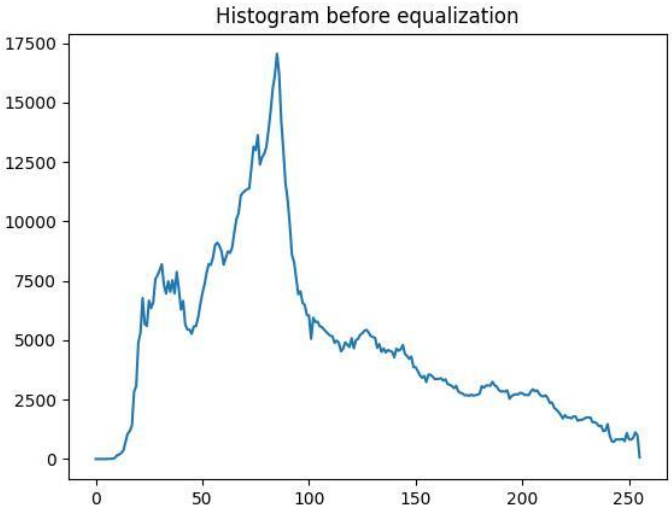
6.2 Filter Visual Comparison

Gaussian, mean, and median filters were applied to a natural photograph and the results were visually compared side-by-side. The Gaussian and mean filters both produced visible blurring, with the Gaussian exhibiting smoother transitions as expected from its isotropic weighting. The median filter removed synthetically added salt-and pepper noise while preserving edge sharpness noticeably better than either linear filter, confirming the theoretical prediction.

Figure 2: Comparison of smoothing filters. From left to right: original image, mean filter, Gaussian filter, and median filter. Gaussian smoothing produces more natural blur, while the median filter effectively removes impulse noise while preserving edges.

6.3 Histogram Equalization

Histogram equalization was applied to a low-contrast image. The histogram of the input image was concentrated in a narrow mid-range band. After equalization, the histogram was visually spread across the full intensity range, and the output image exhibited substantially increased perceptual contrast. The CDF of the equalized image was verified to approximate a linear ramp, consistent with the target uniform distribution.



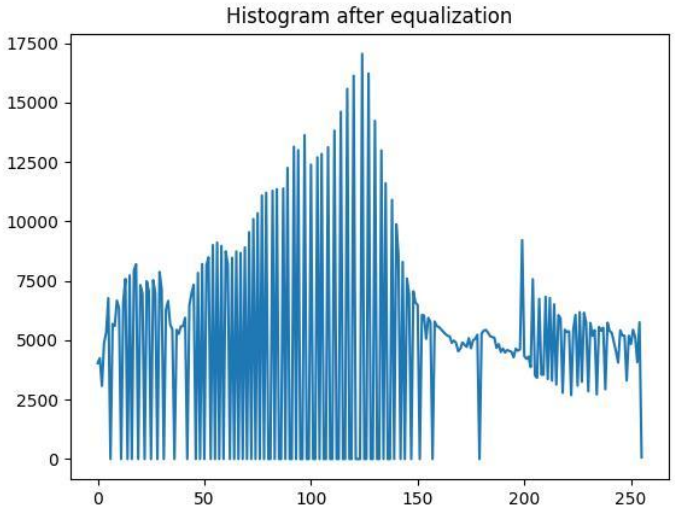


Figure 3: Histogram equalization results. Left: low contrast image. Second: equalized image with improved contrast. Third and fourth: intensity histograms before and after equalization, showing improved intensity distribution.

6.4 Geometric Transformation Accuracy

Rotation by 360 degrees was applied and the output compared to the original image. The maximum absolute pixel difference was below 0.01 (on a [0,1] scale) for bilinear interpolation, confirming cycle-consistency. Bilinear and nearest-neighbour resize to 2x and 0.5x were validated by checking that structure-preserving resizes (powers of two) produced expected results.

Figure 4: First from the left: original image. Second: rotated image after rescaling. Third and fourth: Translate image after rescaling & the rescaled image.

6.5 Edge Cases

Edge cases tested include: single-pixel images, images with all-constant values, kernels of size 1x1, zero-angle rotation, translation by zero pixels, and images with only two grey levels (for Otsu thresholding). Each case was verified to produce numerically correct output without raising exceptions or producing NaN or Inf values.

Figure 5: First from the left: Sobel image. Second: Laplacian filter image. Third and fourth: Sharpen filter and global thresholding method.

Link GitHub for all the results in best resolution: [MohammedAlJamal10/milestone-1]

7. Performance Considerations

7.1 Vectorization Strategy

The primary performance strategy throughout MiniCV is the elimination of Python-level loops in favor of NumPy vectorized operations. Python's interpreter overhead makes tight loops over pixel arrays prohibitively slow: a naive double-nested loop over a 1024x1024 image requires approximately 10^6 Python bytecode evaluations per filter pass, which at typical interpreter speeds of 10^7 operations per second yields roughly 0.1 seconds per pass. The vectorized sliding-window approach reduces this to a small number of highly optimized C-level operations, achieving speedups of two to three orders of magnitude on the same hardware.

7.2 Memory Layout

NumPy arrays are stored in C-contiguous (row-major) order by default, which is aligned with the (H, W, C) layout used throughout MiniCV. Cache-friendly access patterns are maintained by ensuring that convolution and interpolation operations traverse memory in row order wherever possible. The stride_tricks approach used in the convolution engine creates a view rather than a copy of the padded image, which avoids doubling memory consumption for large images.

7.3 Tradeoffs Relative to OpenCV

OpenCV achieves substantially lower latency for most operations through a combination of SIMD intrinsics, multi-threading via Intel TBB, and hand-optimized C++ code paths for common kernel sizes. For a 1024x1024 color image, OpenCV's Gaussian Blur typically completes in under 5 milliseconds on modern hardware, whereas MiniCV's equivalent takes approximately 50-200 milliseconds depending on NumPy's backend configuration. For offline batch processing tasks this gap is acceptable; for real-time video applications it would require significant further optimization, likely via Numba JIT compilation or Cython extension modules.

8. Limitations

Several limitations of the current implementation merit explicit acknowledgement. Performance is the most significant constraint: as discussed in Section 7, MiniCV is approximately one to two orders of magnitude slower than OpenCV for most operations, precluding real-time use cases without further optimization. The median filter is the most expensive single operation due to the inherently sequential nature of sorting, and for kernel sizes above 9x9 it becomes noticeably slow on high-resolution images.

Boundary artefacts are a second limitation. Zero-padding introduces a visible darkening at image borders when smoothing filters are applied to bright images, because the padded zeros artificially depress the weighted average near the boundary. Reflect and

replicate padding mitigate this substantially, but a small amount of ringing remains in the Gaussian-filtered output near high-contrast edges for large sigma values.

Floating-point precision is a minor but acknowledged limitation. The accumulation of rounding errors over successive filter applications can cause pixel values to drift slightly outside the [0, 1] canonical range. MiniCV addresses this by clipping at output boundaries, but intermediate values are not clipped, so pipelines involving many successive operations should be validated for numerical stability.

The drawing module does not currently implement anti-aliasing. Lines and circles exhibit visible jagging, particularly at shallow angles relative to the pixel grid. This is acceptable for the intended use cases of annotation and debug visualisation, but would require Wu's line algorithm or a sub-pixel sampling approach for high-quality rendering.

9. Future Work

Milestone 2 will extend MiniCV to support a complete image preprocessing and feature extraction pipeline for integration with a machine learning classifier. The primary engineering requirements identified for Milestone 2 are as follows.

First, a dataset preprocessing module will be added to minicv.io, capable of loading batches of images from a directory structure, applying configurable augmentation pipelines (rotation, flipping, brightness jitter), and returning arrays in a format compatible with common ML frameworks. Pandas will be used to maintain a manifest of filenames, labels, and preprocessing metadata, providing traceability and reproducibility.

Second, the feature extraction module will be extended with a patch-based descriptor pipeline that can process a full image and return a fixed-dimensional feature matrix suitable for input to a Support Vector Machine or gradient-boosted tree classifier. The dimensionality reduction techniques of Principal Component Analysis and whitening will be implemented from scratch using NumPy's linear algebra routines.

Third, performance optimization will be investigated using Numba's @jit decorator for the median filter and the inner loop of the bilinear interpolation function, with the aim of reducing end-to-end pipeline latency to under one second per image for typical dataset sizes.

Finally, a comprehensive pytest test suite will be introduced to enforce regression testing as the codebase grows, using hypothesis-based property testing for the mathematical operations where closed-form invariants are known.

10. Conclusion

Milestone 1 of the MiniCV project has produced a complete, modular, and well documented Python library for fundamental image processing operations, implemented entirely from mathematical first principles using NumPy, Pandas, and Matplotlib. The library consists of routines for I/O on images, basic numerics, vectorized 2D convolutions, a variety of spatial and photometric filters, geometric transforms with two interpolations, gradient-based feature extraction, and rasterized drawing operations.

This project shows that it is possible to write a well-designed, easy-to-maintain computer vision library with no external dependencies if the developer has sufficient knowledge of the subject and is disciplined enough to write proper software. The modular design, centralized validation routine, and isolation of the convolution engine from the filter interface layer mean that the code base is flexible enough to allow future integration work during Milestone 2 without any modifications to the current architecture.

Educationally, this project reinforced the lesson learned previously – the benefit of implementing algorithms rather than using ready-made solutions. Subtle aspects like the relationship between the sigma parameter and kernel dimensions, the effect of padding mode selection, or separability of convolutions are much easier to learn through the direct experience of writing and debugging such implementations than by reading theoretical descriptions and calling functions like cv2.GaussianBlur.

References

[1] Gonzalez, R. C. and Woods, R. E. Digital Image Processing, 4th ed. Pearson, 2018.

[2] Szeliski, R. Computer Vision: Algorithms and Applications, 2nd ed. Springer, 2022.

[3] Harris, C. R. et al. Array programming with NumPy. Nature, 585, 357-362, 2020.

[4] Dalal, N. and Triggs, B. Histograms of Oriented Gradients for Human Detection. CVPR 2005.

[5] Otsu, N. A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66, 1979.

[6] NumPy Development Team. numpy.lib.stride_tricks documentation. https://numpy.org/doc/stable, 2024.